DESENVOLVIMENTO MOBILE · MÓDULO 05 · TRILHA KOTLIN · NÍVEL INICIANTE

Interfaces com Jetpack Compose

UI declarativa, componentes, layouts, modifiers, listas, Material 3 e temas

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o paradigma declarativo, o layout por restrições e a recomposição.
- Usar os componentes básicos: textos, botões, imagens, campos e cartões.
- Montar telas com `Column`, `Row` e `Box`, alinhamentos e espaçamentos.
- Aplicar `Modifier` entendendo que a ordem importa.
- Exibir listas eficientes com `LazyColumn` e grades.
- Estruturar telas com `Scaffold` e personalizar o tema Material 3.

Roteiro

Neste módulo

- Fundamentos: interfaces declarativas
- Declarativo na prática
- Componentes básicos
- Layouts
- Modifiers
- Listas
- Material 3 e temas
- Fechamento

Fundamentos: interfaces declarativas

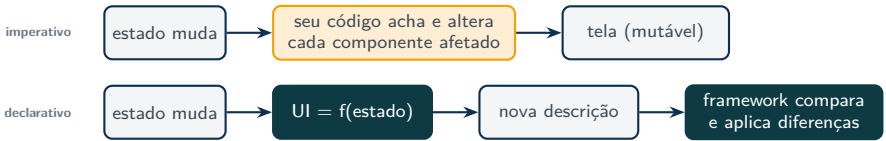
Interfaces imperativas e declarativas

Interface imperativa

O programa cria os componentes uma vez e depois os **altera** passo a passo (`setText`, `setVisibility`). Basta esquecer um deles para a tela mostrar dados errados.

Interface declarativa

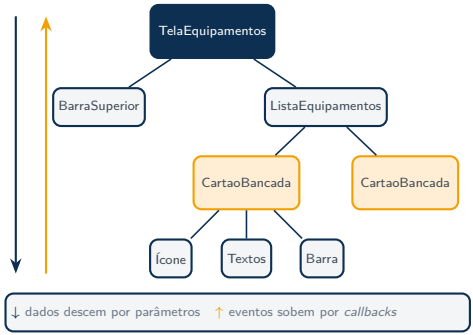
O programa **descreve** como a tela deve ser para cada estado. Quando o estado muda, o framework gera a nova descrição e aplica apenas as diferenças.



Um pouco de história

Views do Android e UIKit do iOS (2008) são imperativos. O React (2013) popularizou a UI declarativa, depois adotada pelo Flutter (2018), SwiftUI (2019) e Jetpack Compose (2021).

Composição: a tela como uma árvore



Composição

Construir interfaces complexas **combinando** componentes pequenos e independentes, em vez de herdar de componentes grandes. Cada componente recebe dados e devolve uma descrição de UI.

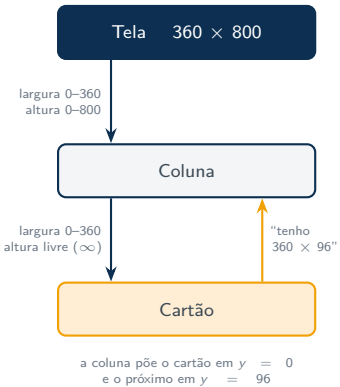
Analogia

Como peças de montar: o cartão é feito de ícone, textos e indicador; a lista, de vários cartões. Trocar uma peça não obriga a refazer as outras.

Um bom componente

Faz uma coisa só, recebe tudo por parâmetro, não sabe de onde vêm os dados e pode ser visto isoladamente em uma *preview*.

Layout por restrições



Restrições (*constraints*)

Intervalo de tamanhos que o pai aceita para o filho: largura e altura mínimas e máximas. **Justa** quando mínimo = máximo; **frouxa** quando o mínimo é zero.

O algoritmo em três frases

Restrições descem: o pai diz os limites. **Tamanhos sobem**: o filho escolhe um tamanho dentro deles. **O pai posiciona**: o filho não sabe onde está.

Uma passada só

Cada nó é medido uma única vez por quadro, o que mantém o layout rápido mesmo com milhares de nós. No Compose, cada `Modifier` ajusta as restrições antes de repassá-las; no Flutter, `Padding` e `SizedBox` fazem esse papel.

Do estado aos pixels: o caminho de um quadro



Quadro e orçamento de tempo

A tela é redesenhada no ritmo do sinal vertical do display (VSync): a 60 Hz cada quadro tem **16,7 ms**; a 120 Hz, só **8,3 ms**. Se o trabalho não couber, o quadro se perde e a animação engasga (*jank*).

Pular fases economiza tempo

Uma cor que anima só precisa de desenho; uma posição que muda, de layout e desenho. Ler o estado na fase mais tardia possível evita repetir as anteriores.

Dica

O *Performance* do Flutter DevTools e o *Layout Inspector* do Android Studio mostram onde o tempo do quadro foi gasto.

Por dentro: descrição, instância e renderização

Papel	Flutter	Compose
Descrição leve, recriada a cada mudança	`Widget`	chamada `@Composable`
Instância duradoura, guarda o estado	`Element` + `State`	composição (`remember`)
Mede, posiciona e desenha	`RenderObject`	`LayoutNode`
Informa leitores de tela	árvore de semântica	árvore de semântica

Analogia

O widget é a planta do arquiteto: barata de redesenhar. O elemento é o apartamento construído, onde mora o estado. O objeto de renderização é a estrutura medida e pintada. Mudar a planta não obriga a demolir o prédio.

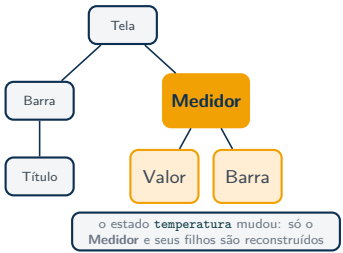
Por que separar

Descrições são criadas aos milhares por segundo, então precisam ser baratas. Medir e desenhar é caro, então os objetos de renderização são **reaproveitados** enquanto possível.

Semântica e acessibilidade

Em paralelo, o framework monta uma árvore de semântica (rótulos, papéis, ações) que o TalkBack e o VoiceOver leem. Ícone sem rótulo fica mudo para quem não enxerga (Módulo 04).

Atualizar sem refazer tudo



Flutter: *rebuild* e reconciliação

`setState` marca o elemento como sujo; no próximo quadro, só ele executa `build`. Filhos de mesmo tipo e chave são reaproveitados e só atualizados.

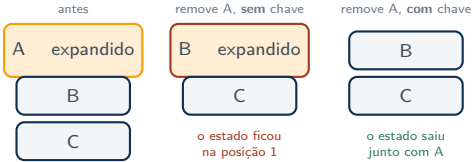
Compose: recomposição inteligente

O Compose sabe quais funções leram cada estado e só executa essas de novo; as que recebem parâmetros iguais aos anteriores são puladas (*skipping*).

Consequência importante

Funções de interface podem rodar muitas vezes, em qualquer ordem: devem ser **rápidas e sem efeitos colaterais**. Rede e banco ficam em APIs de efeito (Módulo 07).

Identidade e chaves



Compose

```
items(itens, key = { it.id }) { eq ->
  ItemEquipamento(eq)
}
```

Flutter

```
ItemEquipamento(
    key: ValueKey(eq.id), eq: eq, onTap: abrir)
```

Chave (*key*)

Identificador estável que diz ao framework qual item é qual entre duas versões da árvore. Sem chave, a correspondência é feita pela **posição**.

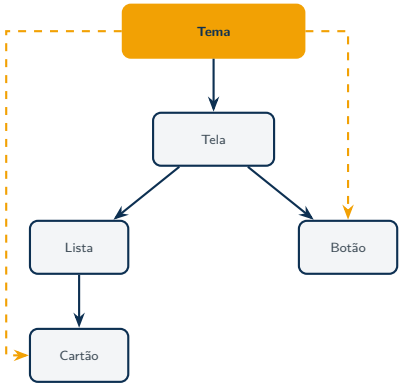
Quando importa

Em listas cujos itens têm estado próprio (expandido, campo digitado, animação) e podem ser inseridos, removidos ou reordenados.

Dica

Use um identificador do domínio, como o `id` do equipamento. Nunca o índice da lista nem um valor aleatório criado a cada reconstrução.

Temas, *tokens* e contexto implícito



tracejado: componentes leem o tema direto do contexto, sem receber parâmetros

Tokens de design

Decisões visuais nomeadas pelo **papel**, não pelo valor: `primary`, `surface`, `titleMedium`. O componente pede "a cor primária"; o tema decide qual é.

Contexto implícito

Tema, idioma e tamanho da tela ficam disponíveis para toda uma subárvore: `InheritedWidget` no Flutter (`Theme.of(context)`), `CompositionLocal` no Compose. Quem lê é atualizado quando o valor muda.

Dica

Com *tokens*, tema escuro e cor dinâmica (Android 12+) funcionam sem mexer nos componentes.

Pixels, densidade e telas de todos os tamanhos



160 dpi: 48 dp = 48 px



480 dpi: 48 dp = 144 px

Pixel independente de densidade

`px = dp × (dpi ÷ 160)`: um botão de 48 dp tem o mesmo tamanho físico em qualquer tela. No Flutter, é o pixel lógico (fator em `devicePixelRatio`).

Texto

Fontes também seguem a escala escolhida pelo usuário: `sp` no Android, `TextScaler` no Flutter.

Responsivo × adaptativo

Responsivo: o mesmo layout se estica ou reorganiza. **Adaptativo**: um layout por classe de tamanho de janela (barra inferior no celular, trilho lateral no tablet). Pense em **espaço disponível**, não em modelo de aparelho.

Android 16 e telas grandes

Com `targetSdk` 36 (exigido pela Play desde agosto de 2026), telas com largura mínima $\geq$ 600 dp ignoram travas de orientação e de redimensionamento; no API 37 não haverá exceção.

Declarativo na prática

Do imperativo ao Compose

Imperativo (Views + XML)

Você cria a árvore de *widgets* e a **modifica** manualmente quando os dados mudam:

```
texto.setText("25 °C")
alerta.setVisibility(View.VISIBLE)
```

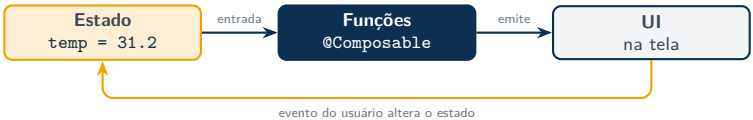
Fácil esquecer algo e deixar a tela inconsistente.

Declarativo (Compose)

Você descreve **como a tela deve ser** para um dado estado. Quando o estado muda, o Compose **redesenha** o que for necessário:

```
Text("$temp °C")
if (temp > 30) Alerta()
```

A tela fica sempre consistente.



UI = f(estado)

Funções `@Composable`

CartaoBancada.kt

```kotlin
@Composable
fun CartaoBancada(
  numero: Int,
  temperatura: Double,
  modifier: Modifier = Modifier
) {
  Column(modifier = modifier.padding(16.dp)) {
    Text(
      text = "Bancada $numero",
      style = MaterialTheme.typography.titleMedium
    )
    Text("%.1f °C".format(temperatura))
    if (temperatura > 30.0) {
      Text("Acima do limite!", color = Color.Red)
    }
  }
}
```

- São funções Kotlin comuns anotadas com `@Composable`.
- Nome em **PascalCase** (são "substantivos").
- Não retornam nada: **emitem** elementos de UI.
- Podem usar `if`, `for` e `when` livremente.
- Recebem um `modifier` opcional como primeiro parâmetro com valor padrão.

Devem ser rápidas e sem efeitos colaterais

O Compose pode chamá-las muitas vezes, em qualquer ordem. Nada de rede, banco ou `Log` pesado dentro delas.

Componentes básicos

Texto, botões, ícones e imagens

Componentes

```
Text(
  text = stringResource(R.string.titulo),
  style = MaterialTheme.typography.headlineSmall,
  maxLines = 1,
  overflow = TextOverflow.Ellipsis
)

Button(onClick = { reservar() }) { Text("Reservar") }
OutlinedButton(onClick = {}) { Text("Cancelar") }
TextButton(onClick = {}) { Text("Detalhes") }
IconButton(onClick = { abrirMenu() }) {
  Icon(Icons.Default.MoreVert, contentDescription = "Menu")
}

Image(
  painter = painterResource(R.drawable.osciloscopio),
  contentDescription = "Foto do osciloscópio",
  contentScale = ContentScale.Crop,
  modifier = Modifier.size(64.dp).clip(CircleShape)
)
```



Acessibilidade

Toda imagem informativa precisa de `contentDescription`; imagens decorativas usam `null`.

Campos de entrada, seleção e cartões

Entrada de dados

```
var descricao by remember { mutableStateOf("") }
var ligado by remember { mutableStateOf(false) }

OutlinedTextField(
  value = descricao,
  onValueChange = { descricao = it },
  label = { Text("Descrição") },
  singleLine = true,
  isError = descricao.isBlank(),
  keyboardOptions = KeyboardOptions(
    keyboardType = KeyboardType.Text)
)

Row(verticalAlignment = Alignment.CenterVertically) {
  Text("Alimentação")
  Switch(checked = ligado, onCheckedChange = { ligado = it })
}

Card(modifier = Modifier.fillMaxWidth()) {
  Text("Conteúdo do cartão", Modifier.padding(16.dp))
}
```

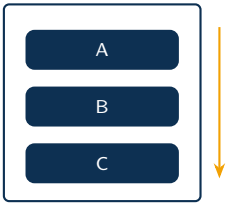
Componentes controlados

O `TextField` **não guarda** o texto: ele mostra o `value` recebido e avisa mudanças por `onValueChange`. Quem guarda o estado é você (Módulo 07).

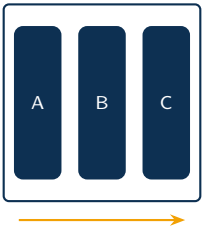
Componente	Uso
`Checkbox`, `Switch`	valores booleanos
`RadioButton`	uma opção entre várias
`Slider`	valor numérico contínuo
`DropdownMenu`	menus e listas suspensas
`AlertDialog`	confirmações
`CircularProgressIndicator`	carregamento

Layouts

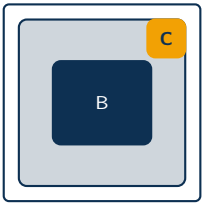
Column, Row e Box



Column
empilha na vertical



Row
alinha na horizontal



Box
sobrepõe (eixo Z)

Arrangement (eixo principal)

`Top`, `Center`, `Bottom`, `SpaceBetween`, `SpaceEvenly`, `spacedBy(8.dp)`.

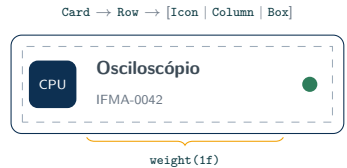
Alignment (eixo cruzado)

Em `Column`: `Start`, `CenterHorizontally`, `End`.
Em `Row`: `Top`, `CenterVertically`, `Bottom`.

Compondo uma tela real

ItemEquipamento.kt

```
@Composable
fun ItemEquipamento(eq: Equipamento, onClick: () -> Unit) {
  Card(onClick = onClick, modifier = Modifier.fillMaxWidth()) {
    Row(
      modifier = Modifier.padding(12.dp),
      verticalAlignment = Alignment.CenterVertically,
      horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
      Icon(Icons.Default.Memory, contentDescription = null)
      Column(Modifier.weight(1f)) {   // ocupa o que sobrar
        Text(eq.descricao, fontWeight = FontWeight.Bold)
        Text(eq.patrimonio,
          style = MaterialTheme.typography.bodySmall)
      }
      Box(
        Modifier.size(12.dp).clip(CircleShape)
          .background(if (eq.disponivel) Verde else Ambar)
      )
    }
  }
}
```



Dica

`Modifier.weight(1f)` distribui o espaço restante dentro de `Row`/`Column`, como o *flex* do CSS.

Dica

`Spacer(Modifier.height(8.dp))` cria espaço vazio entre elementos.

Modifiers

Modifiers: a ordem importa

Mesmos modifiers, ordens diferentes

```
// (a) fundo e DEPOIS espaçamento interno
Text("A", Modifier
  .background(Ambar)
  .padding(16.dp))

// (b) espaçamento e DEPOIS fundo
Text("B", Modifier
  .padding(16.dp)
  .background(Ambar))

// (c) clique antes do padding:
// a área de toque inclui o padding
Text("C", Modifier
  .clickable { /* ... */ }
  .padding(16.dp))
```



(a) fundo inclui o padding (b) fundo só no conteúdo

Como pensar

Os modifiers são aplicados **em cadeia, de fora para dentro**: cada um envolve o resultado dos seguintes.

Categoria	Exemplos
Tamanho	`size`, `fillMaxWidth`, `weight`
Espaço	`padding`, `offset`
Desenho	`background`, `border`, `clip`
Interação	`clickable`, `toggleable`
Semântica	`semantics`, `testTag`

Listas

LazyColumn: listas eficientes

ListaEquipamentos.kt

```kotlin
@Composable
fun ListaEquipamentos(
  itens: List<Equipamento>,
  onItemClick: (Equipamento) -> Unit,
  modifier: Modifier = Modifier
) {
  LazyColumn(
    modifier = modifier,
    contentPadding = PaddingValues(16.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp)
  ) {
    item { Text("Equipamentos (${itens.size})") }
    items(
      items = itens,
      key = { it.patrimonio }      // chave estável!
    ) { eq ->
      ItemEquipamento(eq, onClick = { onItemClick(eq) })
    }
  }
}
```

Lazy = sob demanda

Só compõe os itens visíveis na tela (mais uma margem), como a antiga `RecyclerView`. Uma `Column` com 1000 itens compõe todos de uma vez.

Use `key`

Uma chave estável preserva o estado e as animações de cada item quando a lista é reordenada.

`LazyRow`	lista horizontal
`LazyVerticalGrid`	grade (`GridCells.Adaptive(160.dp)`)
`stickyHeader`	cabeçalhos fixos de seção

Material 3 e temas

`Scaffold`: o esqueleto da tela

TelaEquipamentos.kt

```kotlin
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TelaEquipamentos(itens: List<Equipamento>) {
  Scaffold(
    topBar = {
      TopAppBar(title = { Text("LabMonitor") })
    },
    floatingActionButton = {
      FloatingActionButton(onClick = { /* novo */ }) {
        Icon(Icons.Default.Add, "Adicionar")
      }
    },
    bottomBar = { BarraNavegacao() }
  ) { innerPadding ->
    ListaEquipamentos(
      itens,
      onItemClick = {},
      modifier = Modifier.padding(innerPadding)
    )
  }
}
```



`innerPadding`

Aplique o padding recebido no conteúdo; senão ele fica escondido atrás das barras.

Tema Material 3

ui/theme/Theme.kt

```
private val Claro = lightColorScheme(
  primary = Color(0xFF0D3052),     // azul-petróleo
  secondary = Color(0xFFDE8A1C))  // âmbar
private val Escuro = darkColorScheme(
  primary = Color(0xFF9CC4EA),
  secondary = Color(0xFFE69A1F))
@Composable fun LabMonitorTheme(
  escuro: Boolean = isSystemInDarkTheme(),
  dinamica: Boolean = true,
  content: @Composable () -> Unit
) {
  val ctx = LocalContext.current
  val cores = when {
    dinamica && Build.VERSION.SDK_INT >= 31 ->
      if (escuro) dynamicDarkColorScheme(ctx)
      else dynamicLightColorScheme(ctx)
    escuro -> Escuro
    else -> Claro
  }
  MaterialTheme(cores, content = content)
}
```

Três pilares

Cores (`colorScheme`), **tipografia** (`typography`) e **formas** (`shapes`). Os componentes leem tudo do tema.



Use o tema, não cores fixas

`MaterialTheme.colorScheme.primary` se adapta ao tema escuro e à cor dinâmica do Android 12+; `Color.Red` não. O *Material Theme Builder* gera as cores a partir de uma cor base.

Unidades, recursos e telas adaptáveis

`dp` e `sp`

- `dp` (*density-independent pixel*): tamanhos e espaços. $1\,\text{dp} \approx 1/160$ polegada.
- `sp` (*scale-independent pixel*): fontes; respeita o tamanho de fonte escolhido pelo usuário.

Alvo de toque

Elementos clicáveis precisam de pelo menos **48 dp** $\times$ **48 dp**.

Telas adaptáveis

Use **window size classes** para escolher o layout pelo espaço disponível, não pelo modelo do aparelho:

- **Compacta** (celular em retrato): uma coluna e `NavigationBar`.
- **Média** (dobrável, tablet em retrato): `NavigationRail`.
- **Expandida** (tablet, desktop): lista e detalhe lado a lado.

Dica

`NavigationSuiteScaffold` troca automaticamente a barra inferior pela lateral conforme a largura.

Fechamento

Resumo

Resumo

- Em Compose, a UI é função do estado: descreva a tela e deixe o framework atualizá-la.
- Funções `@Composable` são rápidas, sem efeitos colaterais e recebem um `modifier`.
- `Column`, `Row` e `Box` com `Arrangement`, `Alignment` e `weight` resolvem quase todo layout.
- A ordem dos modifiers define o resultado.
- `LazyColumn` com `key` para listas; `Scaffold` para a estrutura da tela.
- Cores, tipografia e formas vêm do `MaterialTheme`: nada de cores fixas.

Glossário (1/2)

Termo	Significado
UI declarativa	a interface é descrita como função do estado; o framework aplica as mudanças
Composição	montar interfaces combinando componentes pequenos e independentes
`@Composable`	função que emite elementos de interface em vez de devolver um valor
Recomposição	nova execução das funções que leram um estado alterado
Skipping	pular a função cujos parâmetros não mudaram desde a última execução
Restrições	limites mínimos e máximos de largura e altura passados do pai ao filho
`Modifier`	cadeia ordenada que altera tamanho, espaço, desenho e interação
`LazyColumn`	lista que compõe apenas os itens visíveis
Chave (*key*)	identidade estável de um item entre recomposições

Glossário (2/2)

Termo	Significado
`Scaffold`	estrutura padrão de uma tela Material: barras, FAB e conteúdo
***Token* de design**	decisão visual nomeada pelo papel (`primary`, `titleMedium`)
`CompositionLocal`	valor disponível implicitamente para toda uma subárvore
`dp` e `sp`	unidades independentes de densidade; o `sp` segue a escala de fonte
Window size class	classe de largura da janela: compacta, média ou expandida
Jank	quadro perdido por exceder o orçamento de tempo
Árvore de semântica	descrição da tela usada por leitores de tela como o TalkBack

Questões de revisão

1. Qual a diferença entre uma interface imperativa e uma declarativa? Por que a declarativa evita telas inconsistentes?
2. Explique a regra "restrições descem, tamanhos sobem, o pai posiciona" com um exemplo numérico.
3. Quais são as fases de um quadro no Compose e o que cada uma decide?
4. O que é recomposição e quando uma função `@Composable` é pulada?
5. Por que uma função `@Composable` não deve iniciar uma chamada de rede?
6. Por que `Modifier.padding(16.dp).background(c)` é diferente de `Modifier.background(c).padding(16.dp)`?
7. O que acontece em uma `LazyColumn` sem `key` quando o primeiro item, que estava expandido, é removido?
8. Por que usar `MaterialTheme.colorScheme.primary` em vez de `Color(0xFF0D3052)` dentro de um componente?

Exercícios

Exercício 1 — Layout

Reproduza um cartão de "medidor" com título, valor grande, unidade menor ao lado e uma barra de progresso (`LinearProgressIndicator`) colorida conforme a faixa.

Exercício 2 — Modifiers

Crie um botão circular de 56 dp com borda âmbar de 2 dp, fundo azul-petróleo e ícone central. Troque a ordem de `clip` e `background` e explique o resultado.

Exercício 3 — Previews

Escreva `@Preview`s do seu cartão nos temas claro e escuro, com fonte 1,5× e em largura de tablet.

LabMonitor — Etapa 5

Crie a tela **Equipamentos**: `Scaffold` com `TopAppBar`, FAB e `LazyColumn` de `ItemEquipamento` usando os dados de `DadosExemplo`. Aplique o tema com as cores do curso.

Para aprofundar

- Tutorial oficial do Compose: `https://developer.android.com/develop/ui/compose/tutorial`
- Pensando em Compose: `https://developer.android.com/develop/ui/compose/mental-model`
- Layouts básicos: `https://developer.android.com/develop/ui/compose/layouts/basics`
- Modifiers: `https://developer.android.com/develop/ui/compose/modifiers`
- Material 3 em Compose: `https://developer.android.com/develop/ui/compose/designsystems/material3`
- Material Design 3: `https://m3.material.io`

Respostas das questões de revisão

1. No imperativo, o código altera componentes existentes e precisa sincronizá-los a cada evento; no declarativo, descreve-se a tela para cada estado e o framework aplica as diferenças. Há um só caminho do estado à tela, então nada fica esquecido.
2. Na tela de 360 dp, a `Column` passa ao cartão largura de 0 a 360 dp e altura livre; o cartão escolhe 360 $\times$ 96 dp; a coluna o põe em $y = 0$ e o próximo em $y = 96$.
3. Composição (o que mostrar), layout (medir e posicionar) e desenho (pintar); a GPU então gera os pixels. Uma mudança só de cor pode pular as duas primeiras fases.
4. É a nova execução das funções que leram um estado alterado. A função é pulada quando seus parâmetros são estáveis e iguais aos da execução anterior.
5. Porque ela pode rodar muitas vezes e em qualquer ordem: a chamada se repetiria a cada recomposição. Efeitos ficam em `LaunchedEffect` (Módulo 07) ou no `ViewModel`.
6. Os modifiers se aplicam em cadeia, de fora para dentro: no primeiro, o espaçamento vem antes e o fundo cobre só o conteúdo; no segundo, o fundo cobre também o espaçamento.
7. A correspondência é feita pela posição: o estado "expandido" passa para o item que ocupou a posição 0. Com `key = { it.id }`, o estado sai junto com o item removido.
8. O *token* se adapta ao tema escuro, à cor dinâmica e a mudanças da identidade visual; a cor fixa, não.

Fim do Módulo 05

Próximo: Módulo 06 — Kotlin assíncrono: coroutines, dispatchers, Flow e StateFlow.